ЛАБОРАТОРНА РОБОТА № 5
НАВЧАННЯ З ПІДКРІПЛЕННЯМ

Мета: вивчити основи реалізації завдань навчання з підкріпленням у Python.

Хід роботи:

1. Ознайомитися з основними поняттями навчання з підкріпленням.
2. Ознайомитися з можливостями Pygame.
3. Реалізувати поставлені завдання.

.

Теоретичне введення та інструкції до виконання

Використовується бібліотека Python PyGame для створення відеоігор. Ця бібліотека включає кілька модулів для відтворення звуку, графіки, обробки подій тощо. за допомогою Python. За замовчуванням Python не постачається з PyGame як вбудованою бібліотекою. Встановити Pygame можна за допомогою команди:

```
pip install pygame
```

Для початку роботи імпортувати модуль pygame та викликати функцію init() для підготовки модулів pygame до роботи:

```
import pygame
from pygame.locals import *
pygame.init()
```

```python
import pygame, sys
from pygame.locals import *

pygame.init()
DISPLAYSURF = pygame.display.set_mode((300, 100))
pygame.display.set_caption('Простий приклад')
while True: # main game loop
    for event in pygame.event.get():
        if event.type == QUIT:
            pygame.quit()
            sys.exit()
    pygame.display.update()
```

Цикл for, який виконує ітерацію по списку об’єктів Event, які повертає pygame.event.get().

На кожній ітерації циклу for змінній з іменем **event** буде присвоєно значення наступного об’єкта події в цьому списку.

Список об’єктів Event, які повертає pygame.event.get(), буде в тому порядку, у якому відбулися події.

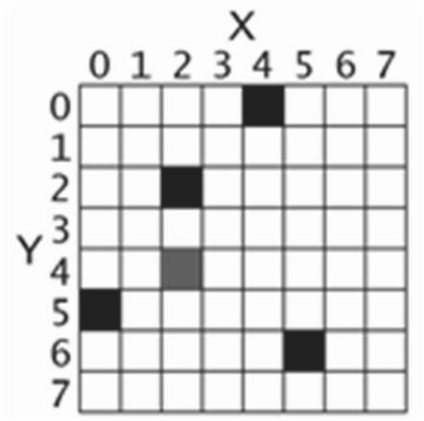
Якщо користувач натиснув кнопку миші, а потім натиснув клавішу клавіатури, об’єкт Event для натискання кнопки миші буде першим у списку, а об’єкт Event для натискання клавіатури буде другим. Якщо жодних подій не сталося, тоді pygame.event.get() поверне порожній список.

Перевіряє, чи дорівнює тип об’єкта Event константі QUIT.

(Оскільки використовували форму оператора імпорту from pygame.locals import *, то потрібно лише ввести QUIT замість pygame.locals.QUIT).

Якщо об’єкт Event є подією виходу, то викликаються функції pygame.quit() і sys.exit().

Координати екрана задаються таким чином:



XY координати:
(4, 0), (2, 2), (0, 5), та (5, 6)

Навчання з підкріпленням

Навчання з підкріпленням — це навчання того, що робити — як співставляти ситуації з діями — щоб максимізувати числовий сигнал винагороди.

Агенту не повідомляється, які дії виконувати, натомість він повинен визначити, які дії приносять найбільшу винагороду, спробувавши їх.

У найбільш цікавих і складних випадках дії можуть вплинути не тільки на негайну винагороду, але й на наступну ситуацію, а через це й на всі подальші винагороди.

Ці дві характеристики — пошук методом проб і помилок і відкладена винагорода — є двома найважливішими відмінними рисами навчання з підкріпленням.

Агент повинен певною мірою сприймати стан свого оточення та вживати дій, які впливають на цей стан.

Агент також повинен мати ціль або цілі, пов'язані зі станом навколишнього середовища.

В інтерактивних задачах часто непрактично отримати приклади бажаної поведінки, які є водночас правильними та репрезентативними для всіх ситуацій, у яких агент повинен діяти.

На незвіданій території, де можна було б очікувати, що навчання принесе найбільшу користь, агент повинен мати можливість вчитися на власному досвіді.

Одним із викликів, які виникають у навчанні з підкріпленням є компроміс між дослідженням та використанням (exploration and exploitation).

Щоб отримати значну винагороду, агент навчання з підкріпленням повинен віддавати перевагу діям, які він пробував у минулому та які виявилися ефективними для отримання винагороди. Але щоб виявити такі дії, він повинен спробувати дії, які він раніше не вибирав.

Агент має використати те, що він уже пережив, щоб отримати винагороду, але він також має дослідити, щоб зробити кращий вибір дій у майбутньому.

Дилема полягає в тому, що ані дослідження , ані використання не можуть здійснюватися виключно без невдачі у виконанні завдання. Агент повинен спробувати різноманітні дії та поступово віддавати перевагу тим, які здаються найкращими.

У стохастичному завданні кожну дію потрібно спробувати багато разів, щоб отримати надійну оцінку очікуваної винагороди.

Окрім агента та середовища, можна виділити чотири основні елементи системи навчання з підкріпленням: політику, сигнал винагороди, функцію цінності та, за бажанням, модель середовища.

Політика (**policy**) визначає спосіб поведінки агента в певний час. Грубо кажучи, політика – це відображення сприйнятого стану навколишнього середовища на дії, які необхідно вжити, перебуваючи в цих станах.

Це відповідає тому, що можна назвати набором правил або асоціацій стимул-реакція. У деяких випадках політика може бути простою функцією або таблицею пошуку, тоді як в інших вона може включати великі обчислення, такі як процес пошуку.

Політика є ядром агента навчання з підкріпленням у тому сенсі, що її однієї достатньо для визначення поведінки. Загалом політики можуть бути стохастичними, вказуючи ймовірності для кожної дії.

Сигнал винагороди (**reward signal**) визначає мету проблеми навчання з підкріпленням. На кожному кроці часу середовище надсилає агенту навчання з підкріпленням єдине число, яке називається винагородою.

Тоді як сигнал винагороди вказує на те, що добре в безпосередньому сенсі, **функція цінності** (**value function**) визначає, що добре в довгостроковій перспективі.

Цінність стану спрощено— це загальна сума винагороди, яку агент може очікувати накопичити в майбутньому, починаючи з цього стану. У той час як винагороди визначають негайну бажаність станів навколишнього середовища, цінності вказують на довгострокову бажаність станів після врахування станів, які, ймовірно, настануть, і винагород, доступних у тих станах.

Наприклад, стан може завжди приносити низьку негайну винагороду, але все ще мати високу цінність, оскільки за ним регулярно йдуть інші стани, які приносять високі винагороди. Або може бути навпаки.

K-armed task -це задача, в якій особа, яка приймає рішення, ітеративно вибирає один з декількох фіксованих варіантів, коли властивості кожного вибору відомі лише частково на момент розподілу

і можуть стати краще зрозумілими з плином часу або шляхом розподілу ресурсів для реалізації варіанту.

Кожна з **k** дій має очікувану або середню винагороду, якщо цю дію вибрано; назвемо це **цінністю** (value) цієї дії.

Дію, обрану на кроці часу t, позначають як A_t, а відповідну винагороду як R_t. Тоді цінність довільної дії **a**, позначене $q_*(a)$, є очікуваною винагородою, якщо вибрано a:

$$q_*(a) \doteq E[R_t|A_t = a]$$

Припускаємо, що цінність дії не відома напевно, хоча можуть бути якісь оцінки. Позначимо оцінену цінність дії a на кроці часу t як $Q_t(a)$. Бажано, щоб значення $Q_t(a)$ було близько до $q_*(a)$.

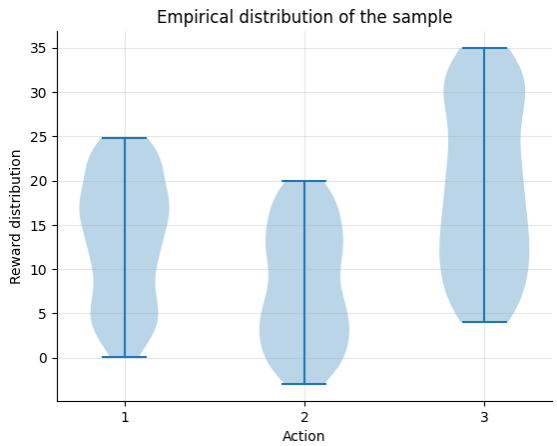
Якщо ви у продовж часу оцінюєте цінність дій, то на будь-якому часовому кроці існує принаймні одна дія, оцінена вартість якої є найбільшою. Їх називають **жадібними діями**.

Коли обирають одну з цих дій, то це означає «використання» поточних знань про значення дій. Якщо натомість обирають одну з нежадібних дій, це означає «досліджувати», оскільки це дає змогу покращити оцінку цінності нежадібної дії. Експлуатація — це правильна річ, щоб максимізувати очікувану винагороду за один крок, але дослідження може принести більшу загальну винагороду в довгостроковій перспективі.

Наприклад, припустимо, що цінність жадібної дії відома з упевненістю, тоді як кілька інших дій оцінюються як майже такі ж хороші, але зі значною невизначеністю. Невизначеність така, що принаймні одна з цих інших дій, ймовірно, насправді є кращою, ніж жадібна дія, але невідомо яка.

Якщо є багато часових кроків, на яких потрібно зробити вибір дій, тоді, можливо, краще дослідити нежадібні дії та виявити, які з них кращі за жадібні.

Винагорода нижча в короткостроковій перспективі під час дослідження, але вища в довгостроковій перспективі, тому що після того, як виявили кращі дії, можна використовувати їх багато разів. Оскільки неможливо одночасно досліджувати та використовувати за допомогою будь-якої однієї дії, говорять про «дилему» між дослідженням та використанням.



Одним із природних способів оцінити це є усереднення фактично отриманих винагород:

$$Q_t(a) \doteq \frac{\sum_{i=1}^{t-1} R_i * \mathbb{I}(A_i = a)}{\sum_{i=1}^{t-1} \mathbb{I}(A_i = a)}$$

Де $\mathbb{I}(A_i = a)$ предикат позначає випадкову змінну, яка дорівнює 1, якщо предикат істинний, і 0, якщо ні.

Якщо знаменник дорівнює нулю, замість цього визначають $Q_t(a)$ як деяке значення за замовчуванням, наприклад 0.

Найпростішим правилом вибору дій є вибір однієї з дій з найвищою оцінкою, тобто однієї з жадібних дій, як визначено раніше.

Якщо є більше ніж одна жадібна дія, то вибір робиться серед них якимось довільним способом, можливо, випадковим чином. Запишемо цей метод вибору жадібних дій як

Жадібний вибір дій завжди використовує поточні знання для максимізації негайної винагороди; він взагалі не витрачає часу на вибір сумнівних дій, щоб побачити, чи дійсно вони можуть бути кращими.

$$A_t \doteq argmax\, Q_t(a)$$

Проста альтернатива полягає в тому, щоб більшу частину часу поводитися жадібно, але час від часу, скажімо, "з невеликою ймовірністю" ε, натомість вибирати випадковим чином серед усіх дій з рівною ймовірністю, незалежно від оцінок цінності дій.

Ці методи, що використовують таке майже жадібне правило вибору дії називають ε-жадібні методи. Перевага цих методів полягає в тому, що в межах, коли кількість кроків збільшується, кожна дія буде відбиратися нескінченну кількість разів, таким чином гарантуючи, що всі $Q_t(a)$ збігаються до $q_*(a)$. Це, звичайно, означає, що ймовірність вибору оптимальної дії збігається до більш ніж $1 - \varepsilon$, тобто до майже напевно.

Додаткові формули для оновлення середніх значень із невеликими постійними обчисленнями, необхідними для обробки кожної нової винагороди.

Враховуючи Q_n і n-ту винагороду, R_n, нове середнє значення всіх n винагород можна обчислити за допомогою

$$Q_{n+1} = Q_n + \frac{1}{n}[R_n - Q_n]$$

A simple bandit algorithm

Initialize, for $a = 1$ to k:
$Q(a) \leftarrow 0$
$N(a) \leftarrow 0$

Loop forever:

$$A \leftarrow \begin{cases} \arg\max_a Q(a) & \text{with probability } 1-\varepsilon \quad \text{(breaking ties randomly)} \\ \text{a random action} & \text{with probability } \varepsilon \end{cases}$$

$R \leftarrow bandit(A)$
$N(A) \leftarrow N(A) + 1$
$Q(A) \leftarrow Q(A) + \frac{1}{N(A)}\left[R - Q(A)\right]$

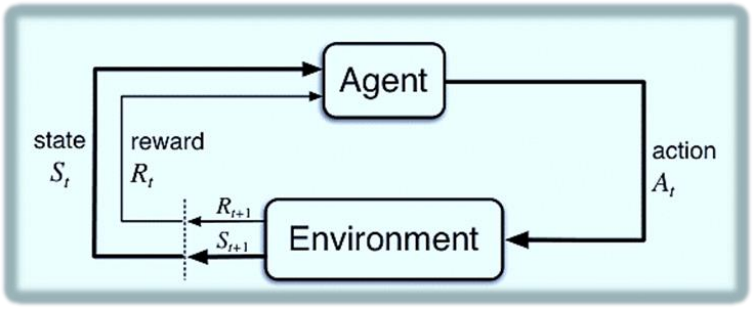
Формальна проблема скінченних марковських процесів прийняття рішень, або скінченних MDP (Markov decision processes)

Ця проблема передбачає оціночний зворотний зв’язок, як у k-armed task, а також асоціативний аспект — вибір різних дій у різних ситуаціях. MDP — це класична формалізація послідовного прийняття рішень, коли дії впливають не лише на негайні винагороди, але й на наступні ситуації чи стани, а через це на майбутні винагороди.

Таким чином, MDP передбачає відкладену винагороду та необхідність компромісу між негайною та відкладеною винагородою. Тоді як у k-armed проблемах оцінювали значення $q_*(a)$ кожної дії **a**, у MDP оцінюють значення $\mathbf{q_*(s, a)}$ кожної дії **a** в кожному стані **s**, або оцінюють значення $\mathbf{v_*(s)}$ кожного стану при виборі оптимальної дії.

Ці величини, що залежать від стану, важливі для точного присвоєння довгострокових наслідків індивідуальним виборам дій.

MDP є математично ідеалізованою формою проблеми навчання з підкріпленням, для якої можна зробити точні теоретичні твердження.



Більш конкретно, агент і середовище взаємодіють на кожному з послідовності дискретних часових кроків, t = 0, 1, 2, 3, ...

На кожному кроці часу **t** агент отримує певне представлення **стану** середовища, $S_t \in S$, і на цій основі вибирає дію $A_t \in A(s)$.

На один крок пізніше, частково як наслідок за виконаної **дії** агент отримує числову **винагороду**, $R_{t+1} \in R \subset \mathbb{R}$, і опиняється в новому стані, S_{t+1}.

MDP і агент разом таким чином створюють послідовність або **траєкторію**, яка починається так:S0, A0, R1, S1, A1, R2, S2, A2, R3, . . .

Мета агента полягає в тому, щоб максимізувати сукупну винагороду, яку він отримує в довгостроковій перспективі.

Якщо послідовність винагород, отриманих після кроку часу **t**, позначається як R_{t+1}, R_{t+2}, R_{t+3}, . . ., тоді прагнуть максимізувати очікувану віддачу, де віддача, позначена G_t, визначається як певна функція послідовності винагороди.

У найпростішому випадку віддача є сумою винагород:

$$G_t \doteq R_{t+1} + R_{t+2} + R_{t+3} + \cdots + R_T \,, \quad (**)$$

де T - кінцевий крок часу. Цей підхід має сенс у додатках, у яких існує природне поняття кінцевого часового кроку, тобто коли взаємодія агента та середовища природним чином розбивається на підпослідовності, так звані **епізоди**, такі як раунд гри, подорожі лабіринтом або будь-яка повторна взаємодія. Кожен епізод закінчується в особливому стані, який називається кінцевим станом, після чого відбувається скидання до стандартного початкового стану або до зразка зі стандартного розподілу початкових станів.

Епізоди закінчуються по-різному, наприклад виграш або програш у грі, наступний епізод починається незалежно від того, як закінчився попередній. Таким чином, можна вважати, що всі епізоди закінчуються в одному термінальному стані, з різними винагородами за різні результати. Завдання з епізодами такого роду називаються **епізодичними**. В епізодичних завданнях іноді потрібно відрізнити набір

усіх нетермінальних станів, позначений S, від набору всіх станів плюс термінальний стан, позначений S+. Час завершення, T, є випадковою величиною, яка зазвичай змінюється від епізоду до епізоду.

З іншого боку, у багатьох випадках взаємодія агент-оточення не розривається природним чином на ідентифіковані епізоди, а триває безперервно без обмежень.

Наприклад, це був би природний спосіб сформулювати поточне завдання керування процесом або застосувати до робота з тривалим терміном служби. Такі завдання називають **безперервними** завданнями.

Застосування формули (**) є проблематичним для безперервних завдань, оскільки останній часовий крок буде T = 1, а віддача, яку ми намагаємося максимізувати, сама по собі може бути нескінченною.

(Наприклад, припустімо, що агент отримує винагороду +1 на кожному часовому етапі.)

Додаткова концепція, це дисконтування. Відповідно до цього підходу агент намагається вибрати дії так, щоб сума дисконтованих винагород, які він отримує в майбутньому, була максимальною. Зокрема, він вибирає A_t, щоб максимізувати очікувану дисконтовану винагороду:

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1},$$

де γ— параметр, $0 \leq \gamma \leq 1$, який називається ставкою дисконту.

Майже всі алгоритми навчання з підкріпленням передбачають оцінку **функцій цінностей** — функцій станів (або пар стан–дія), які оцінюють, наскільки добре для агента перебувати в даному стані (або наскільки добре виконувати певну дію в даному стані).

Відповідно, функції цінностей визначаються щодо конкретних способів дій, які називаються **політиками**.

Формально **політика** — це відображення від станів до ймовірностей вибору кожної можливої дії. Якщо агент дотримується політики π у момент часу t, тоді $\pi(a|s)$ є ймовірністю того, що $A_t = a$, якщо $S_t = s$. Як і p, π є звичайною функцією; «|» в середині $\pi(a|s)$ просто нагадує, що він визначає розподіл ймовірностей на $a \in A(s)$ для кожного $s \in S$. Методи навчання з підкріпленням визначають, як політика агента змінюється в результаті його досвіду.

Функція цінності стану s за політикою π, що позначається $v_\pi(s)$, є очікуваним прибутком, починаючи з s і слідуючи π після цього. Для MDP ми можемо формально визначити v_π за допомогою

$$v_\pi(s) \doteq \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,\middle|\, S_t = s\right], \text{ for all } s \in \mathcal{S},$$

де $\mathbb{E}\pi[\cdot]$ позначає очікуване значення випадкової змінної, враховуючи, що агент дотримується політики π, а t є будь-яким часовим кроком.

Значення термінального стану, якщо таке є, завжди дорівнює нулю.

Функцію v_π називають функцією цінності стану для політики π. Подібним чином визначають значення виконання дії a у стані s згідно з політикою π, позначеною $q_\pi(s, a)$, як очікуваний прибуток, починаючи з **s**, виконуючи дію **a**, а потім дотримуючись політики π :

$$q_\pi(s,a) \doteq \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,\middle|\, S_t = s, A_t = a\right]$$

Використовуючи динамічне програмування:

$$\begin{aligned} v_{k+1}(s) &\doteq \mathbb{E}_\pi[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s] \\ &= \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)\Big[r + \gamma v_k(s')\Big], \end{aligned}$$

Iterative Policy Evaluation, for estimating $V \approx v_\pi$

Input π, the policy to be evaluated
Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
  $\Delta \leftarrow 0$
  Loop for each $s \in \mathcal{S}$:
    $v \leftarrow V(s)$
    $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)\big[r + \gamma V(s')\big]$
    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

$$\begin{aligned} v_{\pi'}(s) &= \max_a \mathbb{E}[R_{t+1} + \gamma v_{\pi'}(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \max_a \sum_{s',r} p(s',r|s,a)\Big[r + \gamma v_{\pi'}(s')\Big] \end{aligned}$$

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
| $\Delta \leftarrow 0$
| Loop for each $s \in \mathcal{S}$:
| $v \leftarrow V(s)$
| $V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)\big[r + \gamma V(s')\big]$
| $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
  $\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a)\big[r + \gamma V(s')\big]$

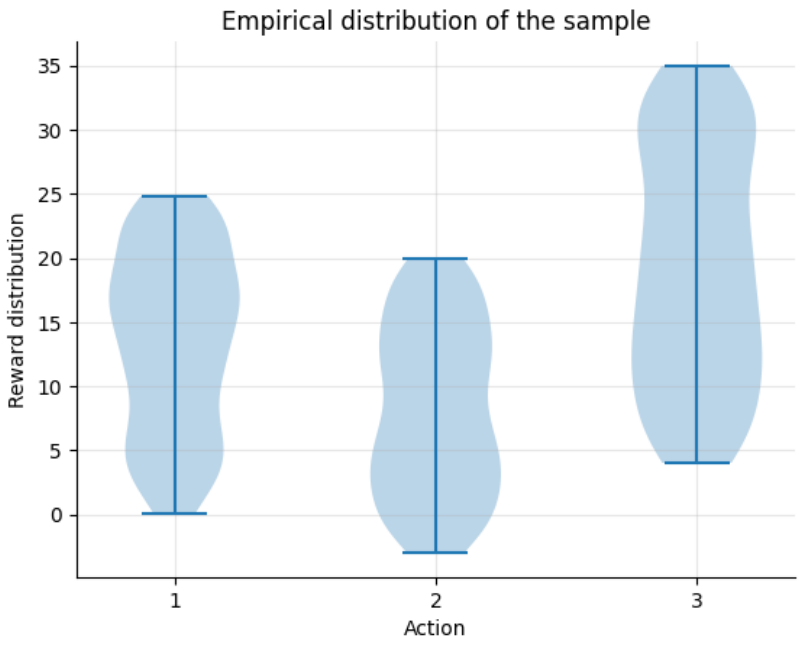
Приклади виконання

Завдання 1. Реалізувати навчання агента вирішенню k-armed task на основі ε-greedy політики

```
import pygame, sys
import numpy as np
from pygame.locals import *
```

Візуалізація розподілу значень для фіксованих варіантів

```
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
d=[np.random.uniform(0.0, 25.0, 250),np.random.uniform(-3.0, 20.0, 250), np.random.uniform(4.0, 35.0, 250)]

fig,ax=plt.subplots()
ax.violinplot(d)
ax.grid(alpha=0.3)
ax.xaxis.set_major_locator(ticker.MultipleLocator(1))
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.set_title('Empirical distribution of the sample')
ax.set_ylabel('Reward distribution')
ax.set_xlabel('Action')
```



Штучний інтелект

Ініціалізація параметрів для графіки

```
# Set up pygame.
pygame.init()
 # Set up the window.
windowSurface = pygame.display.set_mode((500, 400), 0, 32)
pygame.display.set_caption('3-armed task')
# Set up the colors.
BLACK = (0, 0, 0)
WHITE = (255, 255, 255)
RED = (255, 0, 0)
GREEN = (0, 255, 0)
BLUE = (0, 0, 255)
color3_l='#e291fa'
color1_l='#7ffaea'
color2_l='#faab7d'
color3_d='#9d3cba'
color1_d='#0f9992'
color2_d='#ba643c'
# Set up the fonts.
basicFont = pygame.font.SysFont(None, 35)
```

Підготовка блоків для розміщення тексту на формі

```
# Set up the text.
text = basicFont.render('Hello world text!', True, WHITE, BLUE)
textRect = text.get_rect()
textRect.centerx = windowSurface.get_rect().centerx-80
textRect.centery = 300
textRect2 = text.get_rect()
textRect2.centerx = windowSurface.get_rect().centerx-80
textRect2.centery = 350
textRect3 = text.get_rect()
textRect3.centerx = windowSurface.get_rect().centerx+160
textRect3.centery = 350
# Draw the white background onto the surface.
windowSurface.fill('#d5e6bc')
```

```
# Draw a blue circle onto the surface.
coord=[(100,150),(250,150),(400,150)]
colors_l=[color1_l,color2_l,color3_l]
pygame.draw.circle(windowSurface, color1_d, coord[0], 50, 0)
pygame.draw.circle(windowSurface, color2_d, coord[1], 50, 0)
pygame.draw.circle(windowSurface, color3_d, coord[2], 50, 0)

# Get a pixel array of the surface.
pixArray = pygame.PixelArray(windowSurface)
pixArray[480][380] = BLACK
del pixArray

# Draw the window onto the screen.
pygame.display.update()
```

Штучний інтелект

Ініціалізація параметрів для обчислень

```
score=0
d=[np.random.uniform(0.0, 25.0, 250),np.random.uniform(-3.0, 20.0, 250),
np.random.uniform(4.0, 35.0, 250)]
i=0
text3='exploration'
max_iter=200
eps=0.15
q_a=[0,0,0]
n_a=[0,0,0]
r_a=[0,0,0]
```

Код для завантаження графічних елементів та перерисовки екрана

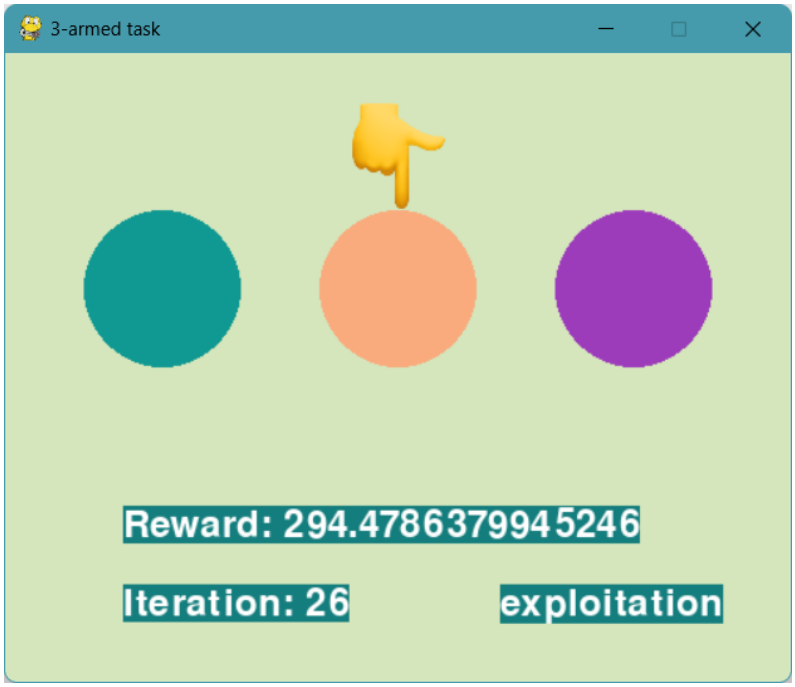
```
def redraw():
    windowSurface.fill('#d5e6bc')
    pygame.draw.circle(windowSurface, color1_d, coord[0], 50, 0)
    pygame.draw.circle(windowSurface, color2_d, coord[1], 50, 0)
    pygame.draw.circle(windowSurface, color3_d, coord[2], 50, 0)
    text = basicFont.render("Reward: "+str(score), True, WHITE, '#147d7d')
    windowSurface.blit(text, textRect)
    text2 = basicFont.render("Iteration: "+str(i), True, WHITE, '#147d7d')
    windowSurface.blit(text2, textRect2)
    windowSurface.blit(basicFont.render(text3,True, WHITE, '#147d7d'), textRect3)
    pygame.display.update()

# create a surface object, image is drawn on it.
imp = pygame.image.load("backhand-index-pointing-down.png").convert_alpha()
imp=pygame.transform.scale(imp, (70,70))
def draw_h(r):
   # Using blit to copy content from one surface to other
    windowSurface.blit(imp, (coord[r][0]-35, 30))
```

Штучний інтелект

Цикл гри: обробка подій, вибір дії, підрахунок винагороди

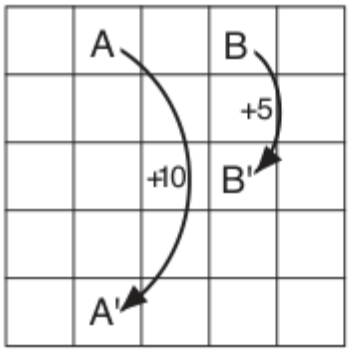
```
while True:
    if i==max_iter:
        break
    redraw()
    pygame.display.update()
    if np.random.rand()<=eps:
        text3='exploration'
        r=np.random.randint(0,3)
    else:
        text3='exploitation'
        r=np.argmax(q_a)
    draw_h(r)
    score+=d[r][i]
    if n_a[r]>0:
        q_a[r]=q_a[r]+(1/n_a[r])*(r_a[r]-q_a[r])
    else:
        q_a[r]=d[r][i]
    n_a[r]+=1
    r_a[r]=d[r][i]

    pygame.draw.circle(windowSurface, colors_l[r], coord[r], 50, 0)
    pygame.display.update()
    i+=1
    #print(score)
    pygame.time.delay(400)
    for event in pygame.event.get():
        if event.type == QUIT:
            pygame.quit()
            sys.exit()
```

Скріншот виконання завдання (максимізація винагороди).

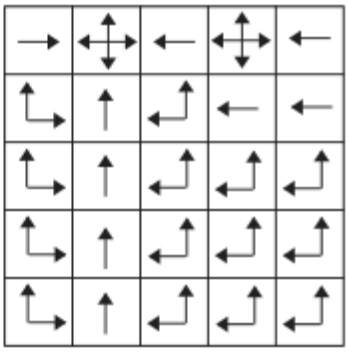


Завдання 2. Вирішення завдання Gridland

Задано поле рис (а), необхідно обчислити функцію цінності станів v* (б) та визначити оптимальну політику π* (в)



22.0	24.4	22.0	19.4	17.5
19.8	22.0	19.8	17.8	16.0
17.8	19.8	17.8	16.0	14.4
16.0	17.8	16.0	14.4	13.0
14.4	16.0	14.4	13.0	11.7



а) б) в)

Агент може переміщатись по клітинкам поля вверх, вниз, вліво, вправо. При переміщенні всередині поля винагорода =0 для всіх клітинок (станів), крім: **A** (винагорода +10, перехід у A’) та **B** (винагорода +5, перехід у B’). При виборі дії, виконання якої виведе агента за межі поля: винагорода -1, агент залишається на попередній позиції. Завдання – максимізація винагороди.

```
import pygame, sys
import numpy as np
from pygame.locals import *
```

Ініціалізація параметрів, обчислення функції цінності та винагороди γ=0.9

```
    tre=0.001
```

Штучний інтелект

```
val=np.zeros((5,5))
rewards=np.zeros((5,5))
rewards[0][1]=10
rewards[0][3]=5
gamma=0.9

def val_s_p(i,j):
    def get_val(i,j):
        #action=up
        v_up=rewards[i][j]+gamma*val[i-1][j] if i>0 else -1+gamma*val[i][j]
        #action=down
        v_dw=rewards[i][j]+gamma*val[i+1][j] if i<4 else -1+gamma*val[i][j]
        #action=left
        v_lt=rewards[i][j]+gamma*val[i][j-1] if j>0 else -1+gamma*val[i][j]
        #action=right
        v_rt=rewards[i][j]+gamma*val[i][j+1] if j<4 else -1+gamma*val[i][j]
        return np.max([v_up,v_dw,v_lt,v_rt])

    if i==0:
        if j==1:
            #point A
            return rewards[i][j]+gamma*val[4][1]
        elif j==3:
            #point B
            return rewards[i][j]+gamma*val[2][3]
        else:
            return get_val(i,j)
    else:
        return get_val(i,j)
```

Обчислення оптимальної цінності станів v*

```
max_tre=0
itter=0
while True:
    itter+=1
    max_tre=0
    for i in range(5):
        for j in range(5):
            val_prime=val_s_p(i,j)
            if max_tre<abs(val_prime-val[i][j]):

                max_tre=abs(val_prime-val[i][j])
                print('i=',i,' j=',j,' delta=',max_tre)
            val[i][j]=val_prime
    print('itter:',itter,' val:',val)
    #val=val1
    print('delta=',max_tre)
    if max_tre<=tre:
        break
```

Приклад створення користувацьких функцій приналежності

Виведення результатів обчислень по ітераціям

Штучний інтелект

```
itter: 2  val: [[ 9.         15.9049     14.31441    10.9049      9.81441   ]
 [ 8.1        14.31441    12.882969   11.5946721  10.43520489]
 [ 7.29       12.882969   11.5946721  10.43520489  9.3916844 ]
 [ 6.561      11.5946721  10.43520489  9.3916844   8.45251596]
 [ 5.9049     10.43520489  9.3916844   8.45251596  7.60726436]]
delta= 9.0
i= 0  j= 0  delta= 5.314410000000002
itter: 3  val: [[14.31441    19.3916844  17.45251596 14.3916844  12.95251596]
 [12.882969   17.45251596 15.70726436 14.13653793 12.72288414]
 [11.5946721  15.70726436 14.13653793 12.72288414 11.45059572]
 [10.43520489 14.13653793 12.72288414 11.45059572 10.30553615]
 [ 9.3916844  12.72288414 11.45059572 10.30553615  9.27498253]]
delta= 5.314410000000002
i= 0  j= 0  delta= 3.138105960899999
itter: 4  val: [[17.45251596 21.45059572 19.30553615 16.45059572 14.80553615]
 [15.70726436 19.30553615 17.37498253 15.63748428 14.07373585]
 [14.13653793 17.37498253 15.63748428 14.07373585 12.66636227]
...
 [17.80096232 19.77921138 17.80129024 16.02116122 14.4190451 ]
 [16.02086609 17.80129024 16.02116122 14.4190451  12.97714059]
 [14.41877948 16.02116122 14.4190451  12.97714059 11.67942653]]
delta= 0.0006855961324170323
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```
... array([[21.9764967 , 24.41877948, 21.97690153, 19.41877948, 17.47690153],
       [19.77884703, 21.97690153, 19.77921138, 17.80129024, 16.02116122],
       [17.80096232, 19.77921138, 17.80129024, 16.02116122, 14.4190451 ],
       [16.02086609, 17.80129024, 16.02116122, 14.4190451 , 12.97714059],
       [14.41877948, 16.02116122, 14.4190451 , 12.97714059, 11.67942653]])
```

Формування матриці оптимальних дій

```
actions=np.empty((5,5,4),dtype=int)

for i in range(5):
    for j in range(5):
        mu=val[i-1][j] if i>0 else 0
        md=val[i+1][j] if i<4 else 0
        ml=val[i][j-1] if j>0 else 0
        mr=val[i][j+1] if j<4 else 0
        m=np.max([mu,md,ml,mr])
        best=np.zeros(4,dtype=int)

        best[0]=1 if mu==m else 0
        best[1]=1 if md==m else 0
        best[2]=1 if ml==m else 0
        best[3]=1 if mr==m else 0
        actions[i,j,:]=best
actions
```

Оптимальна політика

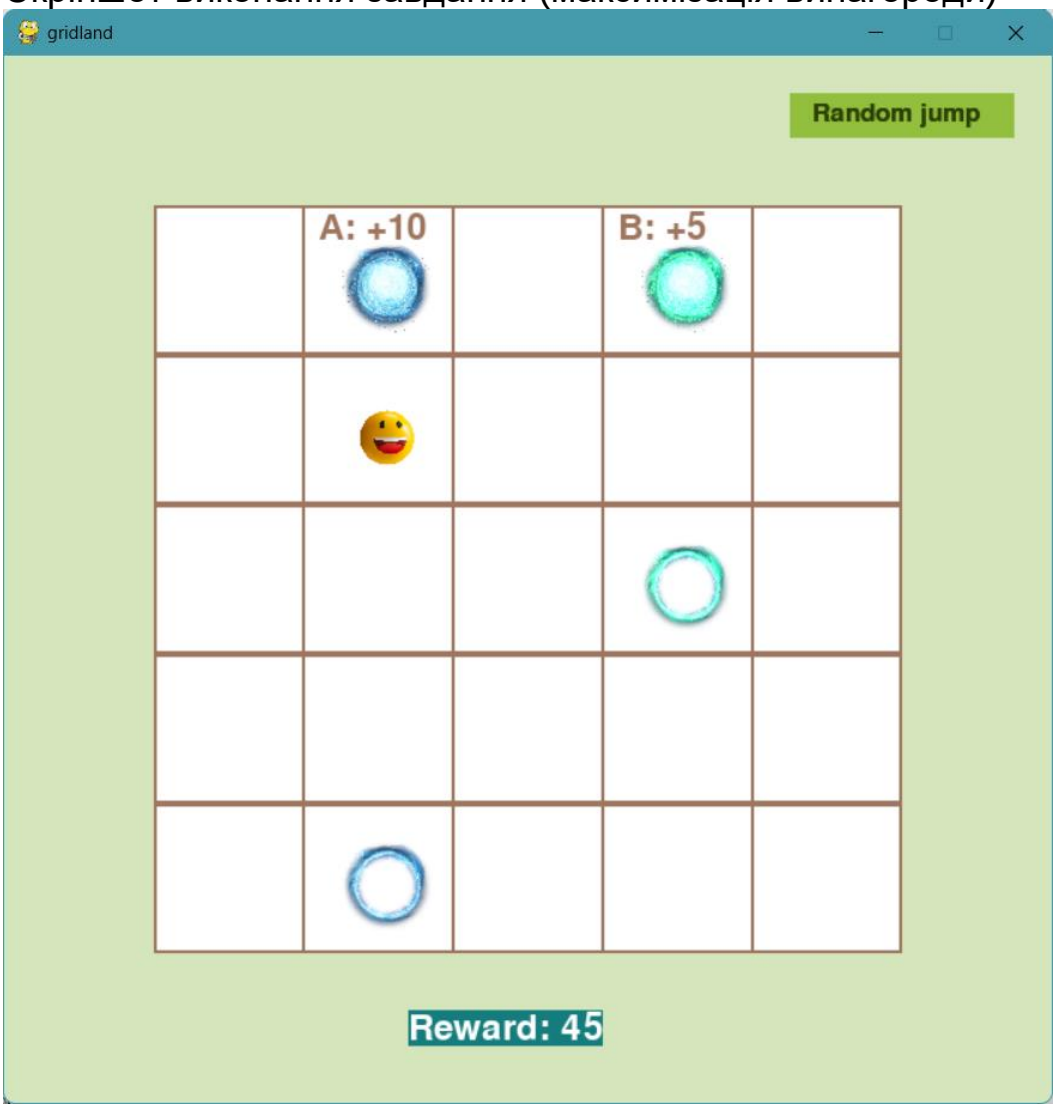
Штучний інтелект

```
moves=np.array(['u','d','l','r'],dtype=str)
for q in range(5):
    for w in range(5):
        mask = np.ma.masked_where(actions[q][w]==0,moves)
        print(np.ma.compressed(mask),' ',end="")
    print("\n")
```

109] ✓ 0.0s

```
['r'] ['d' 'r'] ['l'] ['l'] ['l']

['r'] ['u'] ['u' 'l'] ['l'] ['l']

['r'] ['u'] ['u' 'l'] ['u' 'l'] ['u' 'l']

['r'] ['u'] ['u' 'l'] ['u' 'l'] ['u' 'l']

['r'] ['u'] ['u' 'l'] ['u' 'l'] ['u' 'l']
```

Скріншот виконання завдання (максимізація винагороди)



Варіанти завдань

<table>
<tr><th>№вар</th><th>Завдання 1</th><th>Завдання 2</th></tr>
<tr><td>1</td><td>K=4,
d=[np.random.uniform(-7.0, 15.0, 250),np.random.uniform(-13.0, 10.0, 250), np.random.uniform(1.0, 25.0, 250), np.random.uniform(1.9, 29.0, 250)]
ε=0.3</td><td><table><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td>A
+8</td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td>A`</td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr></table></td></tr>
<tr><td>2</td><td>K=4,
d=[np.random.uniform(-5.0, 18.0, 250),np.random.uniform(-8.0, 10.0, 250), np.random.uniform(0.0, 20.0, 250), np.random.uniform(-5.0, 36.0, 250)]
ε=0.5</td><td><table><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td>A
+7</td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td>A`</td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr></table></td></tr>
<tr><td>3</td><td>K=5,
d=[np.random.uniform(-7.0, 15.0, 250),np.random.uniform(-13.0, 10.0, 250), np.random.uniform(1.0, 25.0, 250) np.random.uniform(0.0, 20.0, 250) np.random.uniform(3.0, 8.0, 250)]
ε=0.4</td><td><table><tr><td></td><td></td><td>A
+9</td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td>A`</td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr></table></td></tr>
<tr><td>4</td><td>K=5,
d=[np.random.uniform(-7.0, 15.0, 250),np.random.uniform(-4.0, 10.0, 250), np.random.uniform(1.0, 21.0, 250) np.random.uniform(5.0, 34.0, 250) np.random.uniform(3.0, 8.0, 250)]
ε=0.2</td><td><table><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td>A
+8</td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td>A`</td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr></table></td></tr>
</table>

<table>
<tr><td>5</td><td>K=4,
d=[np.random.uniform(-5.0, 21.0, 250),np.random.uniform(-8.0, 12.0, 250), np.random.uniform(0.5, 23.0, 250), np.random.uniform(-5.0, 31.0, 250)]
ε=0.3</td><td><table><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td>B
+7</td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td>B`</td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr></table></td></tr>
<tr><td>6</td><td>K=5,
d=[np.random.uniform(-7.0, 18.0, 250),np.random.uniform(-13.0, 11.0, 250), np.random.uniform(1.0, 20.0, 250) np.random.uniform(0.0, 29.0, 250) np.random.uniform(-3.0, 12.0, 250)]
ε=0.25</td><td><table><tr><td></td><td>B
+5</td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td>B`</td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr></table></td></tr>
</table>

Зміст звіту

1. Титульний аркуш
2. Постановка завдання.
3. Короткий опис процедур і функцій, використовуваних при виконанні завдань.
4. Опис реалізації обчислень.
5. Таблиці цінності станів та оптимальної політики.
6. Скріншоти роботи системи.
7. Порівняння винагороди при інших значеннях параметрів γ та ε відповідно до завдань.